

Step-by-Step Testing Guide

Complete walkthrough for running load tests on your REST vs GraphQL research project.

Prerequisites Checklist

Before starting, ensure you have:

- ☐ Docker Desktop installed and running
 - ☐ aps-db repository with seeded data
 - ☐ aps-backend repository with Docker containers running
 - ☐ k6 installed (**k6 version** works)
 - ☐ Node.js installed (**node --version** works)
-

Step 1: Verify Backend is Running

1.1 Check Docker Containers

```
cd aps-backend
docker-compose ps
```

Expected output:

NAME	STATUS
aps-postgres	Up (healthy)
aps-rest-api	Up (healthy)
aps-graphql-api	Up (healthy)

If not all healthy, start them:

```
docker-compose up -d
```

1.2 Test API Endpoints

REST API:

```
curl http://localhost:3001/health
```

Expected: **{"status": "ok", ...}**

GraphQL API:

```
curl http://localhost:3002/.well-known/apollo/server-health
```

Expected: `{"status": "pass"}`

 Step 2: Run Your First Test

Let's run **Scenario 1** for both APIs to verify everything works.

2.1 Test REST API

```
k6 run tests/k6/scenarios/01-simple-list-rest.js
```

What to expect:

- Test runs for ~5 minutes
- You'll see real-time metrics in terminal
- VUs (Virtual Users) ramp up: 10 → 25 → 50 → 100
- Final summary shows latency, throughput, error rate

Success indicators:

-  `http_req_failed` rate < 1%
-  `checks` pass rate > 99%
-  P95 latency < 500ms

2.2 Test GraphQL API

```
k6 run tests/k6/scenarios/01-simple-list-graphql.js
```

What to expect:

- Same duration and load pattern as REST
- Different endpoint (GraphQL)
- Similar metrics reported

2.3 Compare First Results

```
# View summaries
cat tests/results/rest/01-simple-list.json | grep -A 10 "metrics"
cat tests/results/graphql/01-simple-list.json | grep -A 10 "metrics"
```

You should see difference in:

- Response times
- Throughput
- Data transfer sizes



Step 3: Monitor Resources

While a test is running, monitor Docker containers in another terminal.

3.1 Manual Monitoring

```
# In a new terminal window
docker stats aps-rest-api aps-graphql-api aps-postgres
```

What to watch:

- CPU % (should spike during tests)
- Memory usage (should remain stable)
- Network I/O (active during tests)

Press **Ctrl+C** to stop monitoring.

3.2 Automated Monitoring

For production test runs, use the monitoring script:

```
# Monitor for 5 minutes (300 seconds)
./scripts/monitor-resources.sh 300 tests/results/test-resources.csv all
```

This saves data to CSV for later analysis.

Step 4: Run All Scenarios

Now that you've verified everything works, run the complete test suite.

4.1 Run Complete Test Suite

```
./tests/k6/run-all.sh
```

What happens:

1. Creates result directories
2. Runs 6 scenarios for REST (30-40 minutes)

3. Cool-down periods between tests
4. Runs 6 scenarios for GraphQL (30-40 minutes)
5. Saves all results to `tests/results/`

Total duration: ~60-90 minutes

4.2 Monitor Progress

The script shows progress:

```
=====
📊 Scenario: 01-simple-list (rest)
=====

Running k6 test...
[... k6 output ...]

✅ Completed: 01-simple-list (rest)

⌚ Cooling down for 30 seconds...
```

Tips:

- Let it run uninterrupted
- Don't use your computer heavily during tests
- Don't start/stop Docker containers
- Check logs if any test fails

📈 Step 5: Analyze Results

5.1 Generate Comparison Report

```
node scripts/compare-results.js
```

Output:

- Performance comparison tables
- Latency differences
- Throughput comparison
- Data transfer analysis
- Winner for each scenario

5.2 View Individual Scenario Results

```
# View REST Scenario 1 summary
cat tests/results/rest/01-simple-list-summary.json | jq '.metrics'
```

```
# View GraphQL Scenario 4 summary
cat tests/results/graphql/04-nested-dashboard-summary.json | jq '.metrics'
```

5.3 View Resource Usage

```
# View CSV data
head tests/results/rest/01-simple-list-resources.csv

# Or open in Excel/Google Sheets for charts
```

 Step 6: Document Results for Thesis

6.1 Extract Key Metrics

Create a spreadsheet with these columns:

Scenario | API | Avg Latency | P95 Latency | Throughput | Error Rate | CPU Avg | Memory Avg

Example data extraction:

```
# Get P95 latency for REST Scenario 1
cat tests/results/rest/01-simple-list-summary.json | jq
'.metrics.http_req_duration.values["p(95)"]'

# Get throughput
cat tests/results/rest/01-simple-list-summary.json | jq
'.metrics.http_reqs.values.rate'
```

6.2 Create Comparison Tables

Table IV: Performance Metrics Comparison (Example)

Scenario	Metric	REST	GraphQL	Difference
Simple List	P95 Latency	89ms	103ms	+15.7%
Simple List	Throughput	523 rps	498 rps	-4.8%
N+1 Test	P95 Latency	145ms	132ms	-9.0%
...	...	...	...	...

6.3 Create Charts

Use the JSON data to create:

- **Bar chart:** P95 latency comparison (6 scenarios side-by-side)
 - **Line chart:** Throughput over time
 - **Stacked bar:** Resource usage (CPU + Memory)
-

Troubleshooting Common Issues

Issue 1: "k6: command not found"

Solution:

```
# Windows
choco install k6

# Verify
k6 version
```

Issue 2: High error rates in tests

Possible causes:

- Database connection issues
- Containers not healthy
- Too much concurrent load

Solution:

```
# Check container logs
docker-compose logs rest-api
docker-compose logs graphql-api

# Restart services
docker-compose restart

# Reduce load in config files if needed
```

Issue 3: Tests timing out

Solution:

```
# Increase timeout in config files
# Edit tests/k6/config/rest.config.js
export const HTTP_OPTIONS = {
  timeout: '60s', // Increase from 30s
};
```

Issue 4: Booking conflicts in Scenario 5

This is EXPECTED! Booking creation tests will have some conflicts (409 errors) because:

- Multiple VUs try to book same slots
- This is realistic behavior
- Error rate threshold is 5% for write scenarios

Issue 5: Inconsistent results between runs

Solution:

- Run each scenario 3 times
- Use median values
- Ensure system is idle during tests
- Close unnecessary applications

Best Practices for Research

Do:

-  Run tests during low system load (night/weekends)
-  Run each scenario 3 times for statistical validity
-  Document exact conditions (date, time, system state)
-  Save raw results (JSON files) for reproducibility
-  Cool down between tests (30 seconds minimum)
-  Monitor resources during all tests

Don't:

- × Run other heavy applications during tests
- × Modify code between test runs
- × Change Docker resource limits mid-testing
- × Interrupt tests once started
- × Cherry-pick best results (report all runs)

What Success Looks Like

After completing all tests, you should have:

Files (12 test runs):

```
tests/results/
├─ rest/
│   ├─ 01-simple-list-summary.json
│   ├─ 01-simple-list-resources.csv
│   ├─ 02-list-with-relations-summary.json
│   └─ 02-list-with-relations-resources.csv
```

```
| └─ ... (6 scenarios total)
| └─ graphql/
|     └─ 01-simple-list-summary.json
|     └─ 01-simple-list-resources.csv
|     └─ ... (6 scenarios total)
```

Metrics collected:

- ☒ Latency (avg, p50, p95, p99)
- ☒ Throughput (requests/sec)
- ☒ Error rates
- ☒ CPU usage over time
- ☒ Memory usage over time
- ☒ Network I/O

Ready for:

- Chapter 4 (Results) tables
- Performance comparison charts
- Statistical analysis
- Discussion and conclusions

►► Next Steps

1. ☒ Run all tests
2. ☒ Analyze results
3. Create visualizations (charts/graphs)
4. Perform statistical analysis (t-tests, confidence intervals)
5. Write Chapter 4 (Results)
6. Interpret findings for Chapter 5 (Discussion)



Tips for Your Thesis

Chapter 4 (Results) should include:

1. **Methodology recap** (brief reference to Chapter 3)
2. **Test environment details** (Docker resources, k6 version, date)
3. **Raw data tables** (all metrics for all scenarios)
4. **Comparison tables** (REST vs GraphQL side-by-side)
5. **Charts/graphs** (visual comparison)
6. **Statistical analysis** (significance tests)
7. **Key findings** (bullet points of major insights)

Chapter 5 (Discussion) should cover:

1. **Interpretation** of results
2. **Trade-offs** (when to use REST vs GraphQL)

3. **Unexpected findings**
 4. **Limitations** of the study
 5. **Practical implications**
 6. **Future research** directions
-

Good luck with your testing! 🚀

Questions? Review the `LOAD_TESTING.md` for detailed documentation.